



Comandos de LINUX

Sintaxis general: comando [-opciones] argumentos

ls: Lista los elementos que existen en una ubicación del sistema.

- `ls` (o `ls .`) Lista los elementos del directorio actual.
- `ls -l` Listado detallado.
- `ls [ubicación]` Muestra los elementos de la ubicación.

touch: Crea un archivo de texto en la ubicación actual.

- `touch nombre_del_archivo[.ext]`

mkdir: Crea un directorio/carpeta en la ubicación actual.

- `mkdir nombre_directorio`

cd: Permite ubicarse en un directorio.

- `cd [/direccion/que/quiero/acceder]`
- `cd ..` Retrocede en un directorio respecto al actual.
- `cd ../../..` Se pueden encadenar los retrocesos.

pwd: Muestra la ruta actual.

- `pwd`

```
• kosmos_traveler@verdandi:~/CompetitiveProgramming/Cheatsheet fundamentos$ pwd
/home/kosmos_traveler/CompetitiveProgramming/Cheatsheet fundamentos
```

find: Busca un elemento en la dirección indicada, regularmente un archivo.

- `find [direccion] -name cadena_a_buscar`
- `find . -name cadena_a_buscar` Busca en la dirección actual.

clear: Limpia la consola de comandos y procesos previos.

- `clear`

cp: Copia un archivo o directorio de un directorio a otro directorio.

- `cp [elemento_a_copiar] [direccion_donde_se_copia]`
- `cp archivo1[.ext] archivo_copia[.ext]` Copia en el mismo directorio.
- `cp /directorio/cualquiera/archivo[.ext] .` Copia al directorio actual.

mv: Mueve un archivo de un directorio a otro.

- `mv origen/archivo[.ext] destino/archivo[.ext]`
- `mv origen/archivo[.ext] .` Mueve el archivo al directorio actual.
- `mv nombre_actual[.ext] nuevo_nombre[.ext]` Renombra un archivo.

rm: Elimina el archivo o directorio indicado.

- `rm [nombre_directorio/nombre_archivo]`
- `rm -f [nombre_directorio/nombre_archivo]` Fuerza la eliminación del archivo.
- `rm -r [nombre_directorio/nombre_archivo]` Elimina el archivo de forma irreversible.

Comando para compilar C en terminal:

```
gcc nombre_del_archivo.c -o nombre_del_ejecutable[.out/.exe]
./nombre_del_ejecutable[.out/.exe] (Para ejecutar el programa)
```

Lenguaje C

Declaración de bibliotecas: Son bloques de código que contienen funciones importantes, por ejemplo “stdio.h”, que contiene las instrucciones básicas de entrada y salida o “string.h” con las funciones básicas para manejo de cadenas.

```
#include<stdio.h>
#include<stdlib.h>
#include<string.h>
#include<math.h>
```

Entrada y salida: Las funciones “printf()” y “scanf()” reciben cadenas para poder darle formato a la entrada y salida de datos por parte del usuario, esto mediante los formateadores “%d” (enteros) “%f” (flotantes) “%s” (cadenas), entre otros.

```
printf("Ingresa un número: ");
int x=0;
scanf("%d",&x);
```

Variables:
[TipoDato] [nombre];

Arreglos:
[TipoDato] [nombre] [Tamaño];

```
int y = 1700; char c = 'a';
float f = 1.61; long u = 1000;
int arr[10]; char cadena[50];
```

Estructuras condicionales

if(): El condicional if evalúa la expresión que recibe en los paréntesis, si es 0, será falso y si es cualquier otro valor, será verdadero. En caso de ser verdadero permitirá la ejecución del bloque de código que contenga.

else: Debe ir después de un “if” para indicar el caso contrario, es decir, ejecutará su bloque de código si la condición del if previa no se cumple. Se pueden encadenar condiciones combinando “else” e “if”: **else if()**.

switch() - case: Evalúa una expresión numérica definida dentro de los paréntesis, abordando cada caso de forma individual y ejecutando el bloque de código correspondiente al resultado de la evaluación. A esto se suma una sentencia **default** con el bloque de código a ejecutar si ningún caso se cumple.

Estructuras iterativas

while(): while evalúa la expresión de los paréntesis en cada iteración, si es verdadera, ejecutará su bloque de código, si es falsa, lo romperá o no entrará al ciclo en primer lugar.

do-while(): Hace lo mismo que la estructura while, solo que el **do** garantiza que el bloque de código se ejecute al menos una vez.

for(inicio; condición; paso): La estructura for se compone de 3 etapas:

- **Inicio:** Declara o inicializa una variable que se usará en el ciclo.
- **Condición:** Ejecutará el bloque de código mientras la condición se cumpla, se recomienda que se encuentre en términos de la variable que se declaró en el inicio.
- **Paso:** Marca la forma en la cual se cambiará la variable de la que depende el ciclo.

```
x = 2, y = 7;
/* Se pueden encadenar condiciones con
operadores lógicos */
if(x == 0 || y == -100){ // || es OR
    printf("(x) es 0 o (y) es igual a -100\n");
    printf("x: %d - y: %d\n",x,y);
}else if(x == 2 && y==7){ // && es AND
    printf("(y) es 7 y (x) es 2\n");
    printf("x: %d - y: %d\n",x,y);
}else{
    printf("No adiviné tus números :( \n");
    printf("x: %d - y: %d\n",x,y);
}

int a = 10, b = 5;
/* Operadores de comparación:
> (mayor que)
< (menor que)
>= (mayor o igual que)
<= (menor o igual que)
== (igual)
!= (diferente) */
if(a > b){
    printf("(a) es mayor que (b)\n");
    printf("a: %d - b: %d\n", a, b);
}else if(a == b){
    printf("(a) es igual a (b)\n");
    printf("a: %d - b: %d\n", a, b);
}else{
    printf("(a) es menor que (b)\n");
    printf("a: %d - b: %d\n", a, b);
}

int n = 50;
for(int i=0; i<n; i++){
    printf("%d\n",i);
}

char ch = 'p';
switch(ch){
    case 'a':
        printf("Es vocal: %c\n",ch);
        break; // Si no usamos break
                // se ejecutan todos los casos
                // que estén abajo
    case 'e':
        printf("Es vocal: %c\n",ch);
        break;
    case 'i':
        printf("Es vocal: %c\n",ch);
        break;
    case 'o':
        printf("Es vocal: %c\n",ch);
        break;
    case 'u':
        printf("Es vocal: %c\n",ch);
        break;
    default:
        printf("No es vocal: %c\n",ch);
}

x = 72;
while(x > 0){
    printf("%d\n",x);
    x/=2;
}

ch = 'a'-1;
do{
    printf("%c\n",ch);
    ch++;
}while(ch >= a && ch <= 'z');

for(int i=n-1; i>=0; i--){
    printf("%d\n",i);
}
```

Arreglos unidimensionales y bidimensionales

```
int mi_arr[5] = {1, 7, -2, -4, 0};
```

1	7	-2	-4	0
i=0	i=1	i=2	i=3	i=4

```
for(int i=0; i<5; i++){
    printf("%d ",mi_arr[i]);
}
printf("\n");
```

→ 1 7 -2 -4 0

```
int matriz[3][2] = {{-5,2},{1,-2},{0,-3}};
```

	j=0	j=1
i=0	-5	2
i=1	1	-2
i=2	0	-3

```
for(int i=0; i<3; i++){
    for(int j=0; j<2; j++){
        printf("%d ",matriz[i][j]);
    }
    printf("\n");
}
```

→ -5 2
1 -2
0 -3